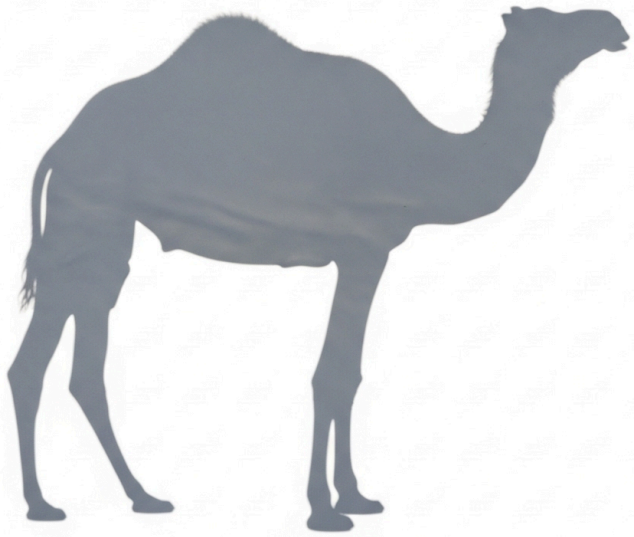
QSWAP

Security Assessment and Code Audit



Mundus

Dec 25th 2025

Rev 1

INTRODUCTION

QSwap is a Uniswap V2-style Automated Market Maker (AMM) DEX for the Qubic blockchain. It enables users to create liquidity pools for QU/Asset pairs, provide liquidity, and swap between QU and various assets.

The contract implements constant product market making ($x \cdot y = k$) with a 0.3% swap fee distributed among

- Shareholders (27%)
- QX (5%)
- Invest & Rewards (3%)
- Burn (1%)
- Liquidity providers (64%)

AUDIT PROCESS

Following considerations have been tested and reviewed by Mundus team:

- Reviewing google test files that are written by SC developer.
- Trying several attack vectors to exploit the SC.
- Ensuring contract logic meets the specifications.
- Thorough line-by-line manual review.

RESULTS

We identified a range of security issues in our assessment, from medium to minor. We recommend fixing them to meet security standards and best practices.

OVERVIEW

1. Project name: **QSWAP**
2. Platform: **QUBIC SC system**
3. Is new SC deployment: **NO**
4. Is upgraded SC: **YES**
5. Project size: Large
6. List of github commits to review: https://github.com/qubic/core/tree/v1.271.0

RESULTS SUMMARY

Total Issues	10
Critical	0
Major	0
Medium	1
Minor	9

AUDIT SCOPE

Filename	SHA	Latest commit
Qswap.h	466767b0346b2006f36aee7a367180aaaed2d85b	fix QSwap
contract_qswap.cpp (gtest)	698f36b5cdc7c1b9fabc42326c166525191ee72f	fix qswap test and adjust epoch for begin_epoch proc

SUMMARY OF FINDINGS

ID	Title	Category	Severity
QSWAP-01	TransferShareManagementRights Fee Handling Issues	Logic	Medium
QSWAP-02	CreatePool Only Allows Asset Issuer Despite Different Input	Logic	Minor
QSWAP-03	No Fee Rate Validation	Logic	Minor
QSWAP-04	Potential Reserve Underflow in Pool State Updates	Logic	Minor
QSWAP-05	Redundant/Duplicate Code in SwapQuForExactAsset	Logic	Minor
QSWAP-06	Typo in Comments	Code Convention	Minor
QSWAP-07	Failed unit test: QuoteTest	Logic	Minor
QSWAP-08	Failed unit test: SwapAssetForExactQu	Logic	Minor
QSWAP-9	Slow traversing for the array `state.mPoolBasicStates`	Logic	Minor
QSWAP-10	Inefficient Pool Slot Recycling	Logic	Minor

FINDINGS

QSWAP-01 TransferShareManagementRights Fee Handling Issues

- File(s) affected: Qswap.h:2244-2282
- Description: The procedure has several issues
  - If fee is insufficient, function returns without refund as at lines 2246-2248.
  - If successful, the fee taken doesn't update to any tracked state variable.
  - The SC always assumes the destination contract always charges 100 Qus, while not all SCs accept fees for Management Right Transfer.
- Impact:
  - Users lost funds due to no refund.
  - Dividends will be incorrect due to missed tracking fees. The retained fees become "zombie funds"—permanently locked in the contract's balance, never distributed to shareholders, or never burned.
  - Users are overcharged for transfers to free contracts.
- Recommendation:
  - Add refund on insufficient fee.
  - Update taken fee to track state variables.
  - Do not assume the fee is 100 Qus. Capture the return value of `qpi.releaseShares` (which indicates the actual fee paid) and use that value to calculate the correct refund for the user.

QSWAP-02 CreatePool Only Allows Asset Issuer Despite Different Input

- **File(s) affected:** Qswap.h:956-960
- **Description:** The CreatePool procedure accepts **assetIssuer** as input but completely ignores it. Instead, it only checks if the invocator issued the asset as at line 956 of Qswap.h.
- **Impact:**
  - Only asset issuers can create pools for their own assets
  - Third parties cannot create pools for popular assets
  - This severely limits DEX functionality
- **Recommendation:** If this is intentional as we only limit the asset issuer to create pools, we should remove the redundant `assetIssuer` from `CreatePool_input`, and only use the `invocator` in the procedure `CreatePool` instead.

QSWAP-03 No Fee Rate Validation

- **File(s) affected:** Qswap.h:2312-2319, 2330-2334
- **Description:** Fee rates are set directly without validation. If rates were accidentally set incorrectly (e.g., during an upgrade), it could break the AMM math.
- **Recommendation:** Add validation constants and checks, example:
  - ASSERT(state.swapFeeRate < QSWAP_SWAP_FEE_BASE);
  - ASSERT(state.shareholderFeeRate + state.investRewardsFeeRate + state.qxFeeRate + state.burnFeeRate <= 100);

QSWAP-04 Potential Reserve Underflow in Pool State Updates

- **File(s) affected:** Qswap.h:1615, 1776, 1963-1967, 2147-2152
- **Description:** Multiple swap functions subtract from reservedQuAmount without explicit underflow checks. The AMM math should prevent this in normal operation, however edge cases with rounding could cause issues.
- **Recommendation:** Use saturating subtraction or add explicit checks, example:

```
locals.totalDeduction = locals.quAmountOut + sint64(locals.feeToShareholders.low)
+ ...;
if (locals.poolBasicState.reservedQuAmount < locals.totalDeduction)
{
    // Handle error
    return;
}
locals.poolBasicState.reservedQuAmount -= locals.totalDeduction;
```

QSWAP-05 Redundant/Duplicate Code in SwapQuForExactAsset

- **File(s) affected:** Qswap.h:1718-1729
- **Description:** The same check appears twice
- **Impact:** Wasted computation, maintenance burden
- **Recommendation:**

QSWAP-06 Typo in Comments

- **File(s) affected:** Qswap.h:632, 690, 753, 822, 986
- **Description:** "solt" should be "slot"
- **Recommendation:**

QSWAP-07 Failed unit test: QuoteTest

- **File(s) affected:** contract_qswap.cpp:314
- **Description:** When calculating the division in `getAmountInTakeFeeFromInToken`, the developer intentionally added 1 for the output, but forgot to do the same for the unit test.
- **Recommendation:** Increase 1 for the unit test expected value at line contract_qswap.cpp:314

QSWAP-08 Failed unit test: SwapAssetForExactQu

- **File(s) affected:** contract_qswap.cpp:588
- **Description:** When calculating the division in `getAmountInTakeFeeFromOutToken`, the developer intentionally added 1 for the output, but forgot to do the same for the unit test.
- **Recommendation:** Increase 1 for the unit test expected value at line contract_qswap.cpp:588

QSWAP-9 Slow traversing for the array **`state.mPoolBasicStates`**

- **File(s) affected:** Qswap.h:331
- **Description:** Every time a user wants to Swap, Add Liquidity, or just check a price, the contractLinear performs a scanning (O(N)) for **`state.mPoolBasicStates`**.
- **Impact:** This affects Compute Tick Limits and Scalability.
- **Recommendation:** Use QPI HashMap object with Hash Map Lookup (O(1)) instead of using the Array object for `state.mPoolBasicStates`.

```
HashMap<Asset, PoolBasicState, QSWAP_MAX_POOL>
mPoolBasicStates;
```

QSWAP-10 Inefficient Pool Slot Recycling

- **File(s) affected:** Qswap.h:1358-1482
- **Description:** The contract uses a fixed-size array `mPoolBasicStates` to track pools. Although there is a low chance this array will be full, it is recommended to have a mechanism to reuse unused pools.
- **Impact:** This affects the slot to remain technically "occupied" even though the pool is empty for a long time with no activity, wasting the array `mPoolBasicStates`.
- **Recommendation:** Should add a mechanism to auto-remove 0 total liquidity after a long time (ie. after a threshold of epochs) with no activity.